FRONTEND UI/UX & CODE EDITOR

Individual Work Plan • Person 1 • Assigned to Jhostan

1. Mission

Jhostan owns the user-facing interface for the Real-Time Collaborative IDE & Debugger. His goal is to build a clear, responsive browser IDE using React and Vite, integrate Monaco Editor, and connect the interface to the collaboration, execution, terminal, persistence, and AI services built by the team.

Core outcome: a user can open a workspace, browse files, edit code, see collaborators, run code, view terminal output, and request AI debugging from one consistent interface.

2. Responsibilities

A. Build the IDE layout

- Set up or use the team's existing React + Vite frontend project.
- Create the main workspace layout: top bar, file explorer, editor area, bottom terminal/output panel, and collaborator area.
- Keep spacing, typography, colors, borders, and component states consistent.
- Support common desktop sizes and make the layout usable at narrower widths.

B. Integrate Monaco Editor

- Mount Monaco Editor in the main editor area.
- Enable syntax highlighting, line numbers, code folding, and language selection where supported.
- Provide clear empty-file and loading states.
- Coordinate with Adarsh (Person 2) to bind Monaco to Yjs using the team's agreed integration.

C. Build workspace and file explorer UI

- Display the workspace file tree and selected file.
- Add controls for creating/opening files if the file APIs are available.
- Show the active file name and unsaved/saved state when supported.
- Call Person 4's agreed APIs rather than implementing database logic in the frontend.

D. Add execution and terminal UI

- Add Run and Debug controls and a clear place for execution results.
- Create the terminal panel and integrate xterm.js with Ashish (Person 3).
- Show standard output, runtime errors, terminal connection state, and useful failure messages.
- Keep browser execution output distinct from remote terminal output.

E. Collaboration and presence UI

- Display active participant avatars or names.
- Render remote cursors and selections using the presence data supplied by Adarsh.
- Show connection/reconnection status.
- Avoid forcing one participant's editor viewport or scrolling onto other participants.

F. AI debugger UI and integration

- Add a Debug / Explain Error action.
- Send the current code and relevant error context to Person 4's AI endpoint using the agreed request format.
- Display the AI explanation and suggested fix in a readable panel.

- Require user review before a suggested fix is applied to the shared document.

3. Suggested Component Structure

Adapt this to the team's existing repository. Avoid duplicating files already created by teammates.

```
src/
  components/
    TopBar.jsx
    FileExplorer.jsx
    EditorPane.jsx
    TerminalPanel.jsx
    OutputPanel.jsx
    Collaborators.jsx
    AIDebuggerPanel.jsx
  pages/
    Workspace.jsx
  services/
    collaborationClient.js   # interface supplied with Person 2
    executionClient.js       # interface supplied with Person 3
    workspaceApi.js          # API contract supplied with Person 4
    aiDebuggerApi.js         # API contract supplied with Person 4
```

4. Suggested Work Sequence (24-hour hackathon)

Time	Tasks	Milestone
Hours 0–3	Agree on layout and API contracts; scaffold React + Vite and build the main IDE shell.	UI skeleton runs.
Hours 3–6	Integrate Monaco and build file explorer layout.	Editor visible.
Hours 6–9	Connect Monaco to Adarsh's Yjs provider; test with two browser sessions.	Collaborative editor works.
Hours 9–12	Add Run, output, and terminal panel; integrate Ashish's execution interface.	Execution UI connected.
Hours 12–15	Connect file save/load APIs and display participant presence.	Workspace flow works.
Hours 15–18	Connect AI debugger panel to Person 4's endpoint.	AI result visible.
Hours 18–21	Polish responsive layout, loading/error states, and keyboard usability.	UI feels consistent.
Hours 21–24	Run integrated tests with all members; fix visual and connection bugs.	Demo-ready UI.

5. Interfaces to Agree on Early

- **Editor + collaboration:** Adarsh provides the Yjs provider/room connection details and awareness data shape.
- **Execution + terminal:** Ashish provides the Run function contract, terminal WebSocket URL, and output/error format.
- **Workspace files:** Person 4 provides endpoint paths, request/response shapes, and saved-state behavior.
- **AI debugging:** Person 4 provides the debug endpoint and expected code/error payload.
- **Shared repository:** agree on component ownership and avoid simultaneous edits to the same files; use feature branches and pull requests.

6. Acceptance Checklist

- ■ Frontend starts using documented instructions.
- ■ IDE layout contains editor, file explorer, terminal/output, and collaboration indicators.
- ■ Monaco can open and edit a file.
- ■ Two users can edit the same file through the collaboration integration.
- ■ Run action displays output and errors returned by the execution module.
- ■ Terminal panel connects to Ashish's service and displays streamed output.
- ■ Workspace files can be opened and saved through Person 4's APIs.

- ■ AI debugging response appears in the UI and proposed fixes require user confirmation.
- ■ Connection, loading, empty, and error states are understandable.
- ■ The full frontend works in the team's end-to-end demo.

7. Important Notes

- • Jhostan owns the overall UI and frontend integration, but should not duplicate the backend logic assigned to the other members.
- • Use mock data only temporarily while teammates build APIs; replace it with the agreed service calls before the demo.
- • Do not put database credentials or OpenAI API keys in browser code.
- • Build a usable editor and shared editing flow before spending too much time on visual polish.
- • Keep browser execution results separate from cloud terminal output so users understand where code ran.

Definition of done: the integrated frontend lets a user open a workspace, edit collaboratively, run code, see terminal output, save files, and request AI debugging through the team's agreed APIs.